BUSINESS PROPOSAL - STOCKY AI

Team Grabbeez

UMHackathon 2026

Table of Contents

1. Executive Summary. 3

2. Deployment Overview.. 4

2.1 Deployment Trigger Conditions. 4

2.2 High-Level Pipeline Flow.. 4

3. Deployment Strategies. 6

3.1 Rolling Deployment (Currently Implemented) 6

3.2 Blue/Green Deployment (for larger scale) 7

3.3 Canary Deployment 7

3.4 Strategy Comparison. 8

4. Technical Deployment Details. 9

4.1 GitHub Actions Workflow.. 9

4.2 Docker Build Process. 9

4.3 Amazon ECR (Elastic Container Registry) 10

4.4 Amazon ECS (Elastic Container Service) 10

4.5 AWS Secrets and Variables Configuration. 11

5. Deployment Risks. 12

5.1 Application Risks. 12

5.1.1 Broken Build or Failing Tests. 12

5.1.2 Runtime Failures After Deployment 12

5.1.3 Incompatible Database Schema Changes. 12

5.2 Infrastructure Risks. 13

5.2.1 Credential Exposure or Compromise. 13

5.2.2 ECR Image Vulnerabilities. 13

5.2.3 ECS Task Placement Failures. 13

6. Deployment Considerations. 14

6.1 Image Tagging Strategy. 14

6.2 Rollback Procedure. 14

6.3 Environment Promotion. 14

6.4 Observability and Monitoring. 15

6.5 Secrets Management 15

7. Scalability and Performance. 16

7.1 Horizontal Scaling with ECS Service Auto Scaling. 16

7.2 Fargate vs EC2 Launch Type. 16

7.3 Load Balancing. 17

7.4 Statelessness and Data Architecture. 17

7.5 CI/CD Pipeline Scalability. 17

8. Post-Deployment Checklist 18

9. Glossary. 19

1. Executive Summary

This Deployment Plan defines the end-to-end strategy, procedures, and considerations for deploying the containerised application to Amazon Web Services (AWS) using Docker, Amazon Elastic Container Registry (ECR), and Amazon Elastic Container Service (ECS). The pipeline is fully automated via GitHub Actions, triggering on every successful merge to the main branch.

The goal of this document is to provide a clear, reproducible deployment blueprint that minimises downtime, reduces human error, and ensures that the production environment can be reliably updated and rolled back if necessary. It covers deployment architecture, strategy selection, infrastructure components, risk management, and scalability planning — all tailored to the GitHub Actions CI/CD workflow defined in the project repository.

This document is intended for the engineering and DevOps teams participating in the hackathon and should serve as a living reference throughout the deployment lifecycle.

2. Deployment Overview

The deployment pipeline is triggered automatically whenever code is pushed to the main branch and the preceding test job passes successfully. The workflow authenticates to AWS, builds a Docker image, tags it, pushes it to ECR, and instructs ECS to deploy the latest image to the running service. This fully automated flow ensures that every merged commit is production-ready and traceable.

2.1 Deployment Trigger Conditions

- Event type must be a push (not a pull request or manual dispatch).
- The target branch must be refs/heads/main — feature branches and staging branches do not trigger deployment.
- The test job defined in the workflow must complete with a success status. If any test fails, the deploy job is skipped automatically.
- AWS credentials (AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_ACCOUNT_ID) must be present and valid in GitHub Secrets.
- The ECR repository and ECS cluster/service referenced by the workflow variables must already exist in the target AWS region.

2.2 High-Level Pipeline Flow

Stage	Description
Step 1	Developer pushes code to the main branch on GitHub.
Step 2	GitHub Actions detects the push event and evaluates the workflow conditions.
Step 3	The test job runs the project's test suite. On failure, pipeline halts.
Step 4	On test success, the deploy job is triggered.
Step 5	AWS credentials are configured via the configure-aws-credentials action.

Step 6	The amazon-ecr-login action authenticates Docker to the ECR private registry.
Step 7	Docker builds the application image from the repository Dockerfile.
Step 8	The image is tagged with the ECR repository URI and the latest tag.
Step 9	The tagged image is pushed to the ECR private repository.
Step 10	ECS is updated (via CLI or task definition update) to pull and run the new image.

3. Deployment Strategies

Choosing the right deployment strategy is critical to balancing deployment speed, system availability, and risk. Below are the most relevant strategies evaluated for this project, including the currently implemented approach and recommended alternatives for production hardening.

3.1 Rolling Deployment (Currently Implemented)

Rolling deployment is the strategy that AWS ECS uses by default when a service update is triggered. In this model, ECS gradually replaces running task instances with new ones, maintaining a minimum healthy percentage of tasks throughout the update. The existing workflow pushes a new image to ECR and triggers an ECS service update, which internally performs a rolling replacement.

How it works:

- ECS stops a subset of the old tasks (based on the configured minimum healthy percentage and maximum percent settings).
- New tasks are launched using the updated task definition pointing to the new Docker image.
- ECS waits for the new tasks to pass health checks before removing more old tasks.
- The process continues until all tasks are running the new version.

Advantages:

- Zero-downtime in most cases, as healthy old tasks continue serving traffic during the transition.
- No additional infrastructure cost — the rollout happens within the existing cluster capacity.
- Simple to configure and requires no changes to the current GitHub Actions workflow.

Limitations:

- Both old and new versions run simultaneously during the rollout window, which can cause issues if the API contract between versions is incompatible.
- Rollback requires re-deploying the previous task definition, which adds time.
- Not ideal for database migrations that are not backward-compatible.

3.2 Blue/Green Deployment (for larger scale)

Blue/Green deployment maintains two identical production environments — the Blue environment (currently live) and the Green environment (the new version). Traffic is switched from Blue to Green only after the Green environment has been fully provisioned and validated. AWS CodeDeploy integrates natively with ECS to automate this strategy.

How it works:

- The new version is deployed to the Green environment (a new ECS task set) while the Blue environment continues to serve 100% of production traffic.
- Automated health checks and optional manual approval gates validate the Green environment.
- Traffic is shifted from Blue to Green, either all at once or gradually using weighted routing via an Application Load Balancer.

- The Blue environment is kept alive for a configurable retention period to enable instant rollback.

Advantages:

- Instant, zero-risk rollback by simply switching traffic back to Blue.
- Full validation of the new version under production conditions before it receives live traffic.
- Clean separation between old and new versions — no mixed-version state.

Limitations:

- Requires double the infrastructure resources during the transition window.
- Additional configuration overhead — CodeDeploy deployment groups, load balancer listener rules, and target groups must be set up.
- Slightly more complex GitHub Actions integration requiring CodeDeploy API calls.

3.3 Canary Deployment

Canary deployment is a progressive rollout strategy where a small percentage of production traffic is shifted to the new version first. The system is monitored closely, and if no anomalies are detected, traffic is gradually increased until the new version handles 100% of requests.

How it works:

- Deploy the new image to a small subset of ECS tasks — for example, 5% or 10% of the total task count.
- Route a matching percentage of incoming traffic to the new tasks using weighted target groups on the Application Load Balancer.
- Monitor error rates, latency, and custom application metrics using Amazon CloudWatch.

- If metrics remain within acceptable thresholds, incrementally increase the traffic percentage in stages (e.g. 10% → 25% → 50% → 100%).
- If anomalies are detected at any stage, traffic is immediately shifted back to the stable version.

Advantages:

- Minimises the blast radius of a faulty deployment — only a small fraction of users are affected if the new version has issues.
- Real-world validation with live traffic before full rollout.
- Works well in combination with feature flags for even more fine-grained control.

Limitations:

- Requires robust monitoring and alerting infrastructure to be meaningful.
- More complex to orchestrate — needs weighted routing, automated metric evaluation, and rollback triggers.
- Not built into the current GitHub Actions workflow and requires additional tooling such as AWS CodeDeploy or a custom promotion script.

3.4 Strategy Comparison

Strategy	Summary
Rolling	ECS default. Low complexity, no extra cost. Old + new run simultaneously. Suitable for the current hackathon context.
Blue/Green	Two full environments. Instant rollback. Ideal for production systems with zero-tolerance downtime requirements.
Canary	Gradual traffic shift. Requires monitoring infrastructure. Best for high-traffic, risk-sensitive production workloads.
Recreate	Tear down all old tasks, deploy new ones. Maximum downtime. Only suitable for non-production or dev environments.

Recommendation: For the hackathon, the rolling deployment strategy currently in use is appropriate. For a production system, Blue/Green deployment using AWS CodeDeploy provides the best balance of safety and simplicity on ECS.

4. Technical Deployment Details

This section covers the infrastructure components, configuration requirements, and technical implementation details underpinning the deployment pipeline.

4.1 GitHub Actions Workflow

The CI/CD pipeline is defined as a GitHub Actions workflow with two jobs: test and deploy. The deploy job carries a dependency on the test job via the needs: test directive and is gated by a conditional that checks both the event type and the target branch. This ensures that only code merged to main — and only after passing tests — triggers a production deployment.

Key workflow directives:

- runs-on: ubuntu-latest — each job executes on a fresh GitHub-hosted Ubuntu runner, ensuring a clean build environment with no state carryover between runs.
- uses: actions/checkout@v4 — checks out the repository code at the commit that triggered the workflow, making the Dockerfile and application source available to the runner.
- uses: aws-actions/configure-aws-credentials@v4 — injects the AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY from GitHub Secrets as environment variables used by subsequent AWS CLI and SDK calls.
- uses: aws-actions/amazon-ecr-login@v2 — authenticates the Docker daemon on the runner to the private ECR registry, enabling docker push commands to succeed without manual login steps.

4.2 Docker Build Process

The Docker build step creates a container image from the Dockerfile at the root of the repository. The image is tagged using the ECR_REPOSITORY variable, and the build context is the entire repository working directory. The runner's Docker engine handles layer caching; however, because GitHub Actions runners are ephemeral, the build cache does not persist across runs unless explicitly configured using Docker layer caching actions.

- The image is first built locally on the runner: docker build -t $ECR_REPOSITORY .
- The image is then re-tagged with the full ECR URI before pushing. The tag format is: <account_id>.dkr.ecr.<region>.amazonaws.com/<repository>:latest.
- Using :latest as the tag means each deployment overwrites the previous image reference. For auditability and rollback, it is recommended to also tag images with the Git commit SHA (e.g. :sha-abc1234) in addition to :latest.

4.3 Amazon ECR (Elastic Container Registry)

Amazon ECR is a fully managed, private Docker container registry that stores, encrypts, and manages Docker images. The pipeline pushes the built image to an ECR repository, from which ECS pulls the image when launching new task instances.

Configuration and operational notes:

- The ECR repository name is provided via the vars.ECR_REPOSITORY GitHub Actions variable, keeping the repository name configurable without hardcoding it in the workflow file.
- ECR repositories are region-specific. The region is sourced from vars.AWS_REGION and must match the region where the ECS cluster is deployed.

- ECR supports image scanning for OS-level vulnerabilities. Enabling scan-on-push is strongly recommended to catch known CVEs before images reach production.
- Lifecycle policies should be configured on the ECR repository to automatically expire untagged images and retain only the most recent N tagged images, preventing unbounded storage costs.
- Image encryption is handled by AWS at rest using AWS Key Management Service (KMS). For compliance-sensitive workloads, a customer-managed KMS key can be specified.

4.4 Amazon ECS (Elastic Container Service)

Amazon ECS is the container orchestration platform that runs and manages the Docker containers in production. ECS operates on either the Fargate (serverless) or EC2 launch type. The service continuously reconciles the desired state (number of running tasks, task definition version) with the actual state of the cluster.

Core ECS concepts relevant to this deployment:

- Task Definition — a blueprint describing the container(s) to run, including the ECR image URI, CPU and memory allocations, port mappings, environment variables, IAM task role, and logging configuration. Each deployment creates a new task definition revision pointing to the updated image.
- Service — an ECS construct that maintains a desired count of simultaneously running tasks from a specified task definition. The service orchestrates the rolling update during deployment and restarts failed tasks automatically.
- Cluster — a logical grouping of infrastructure resources (Fargate capacity or EC2 instances) on which tasks are scheduled and executed.
- Service Auto Scaling — ECS services can be configured to scale the task count automatically based on CloudWatch metrics such as CPU utilisation, memory utilisation, or custom application metrics published via the CloudWatch API.

4.5 AWS Secrets and Variables Configuration

The workflow references several GitHub Secrets and Variables that must be pre-configured in the repository settings before the pipeline can execute successfully. These are not stored in the repository source code and must be managed through the GitHub UI or GitHub CLI.

Name	Description
AWS_ACCESS_KEY_ID	GitHub Secret. AWS IAM access key ID for programmatic authentication. Must have permissions for ECR push and ECS service update.
AWS_SECRET_ACCESS_KEY	GitHub Secret. AWS IAM secret access key paired with the access key ID. Must be rotated regularly.
AWS_ACCOUNT_ID	GitHub Secret. The 12-digit AWS account ID used to construct the full ECR image URI.
AWS_REGION	GitHub Variable. The AWS region where ECR and ECS resources are deployed (e.g. ap-southeast-1).
ECR_REPOSITORY	GitHub Variable. The name of the ECR repository (e.g. my-app). Used in the docker build tag and push commands.

Important: AWS IAM best practice requires that the access key used by GitHub Actions follows the principle of least privilege. The IAM user or role should have only the permissions required — specifically ecr:GetAuthorizationToken, ecr:BatchCheckLayerAvailability, ecr:PutImage, ecr:InitiateLayerUpload, ecr:UploadLayerPart, ecr:CompleteLayerUpload, and the relevant ecs:UpdateService and ecs:RegisterTaskDefinition actions.

5. Deployment Risks

All deployment activities carry inherent risk. This section identifies the primary risks associated with the current pipeline and provides mitigation strategies for each.

5.1 Application Risks

5.1.1 Broken Build or Failing Tests

If the Docker image build fails due to a dependency issue, a build-time error, or an incompatible base image update, the pipeline will stop and no new image will be pushed. However, the ECS service will continue running the last successfully deployed image.

- **Pin base image versions in the Dockerfile (e.g. FROM node:20.11-alpine rather than FROM node:latest) to prevent unexpected breaks caused by upstream image updates.:** Mitigation
- **Add a docker build --no-cache step to the pipeline periodically to catch issues that are masked by layer caching.:** Mitigation

5.1.2 Runtime Failures After Deployment

A new image may build and push successfully but fail when ECS attempts to start the container — for example, due to a misconfigured environment variable, a missing secret, or an application startup crash.

- **Configure ECS health checks on the task definition or load balancer. ECS will stop launching new tasks if the health check fails, and the rolling update will stall rather than taking down all running tasks.:** Mitigation
- **Set a minimum healthy percent of at least 50% on the ECS service to ensure that old tasks remain running if new tasks fail to become healthy.:** Mitigation
- **Enable container-level health checks in the Dockerfile using the HEALTHCHECK directive.:** Mitigation

5.1.3 Incompatible Database Schema Changes

Deploying application code that depends on a new database schema while old instances are still running with the old schema can cause runtime errors in both old and new tasks during the rolling update window.

- **Follow the expand-contract migration pattern: first deploy a schema migration that is backward-compatible with the old code, then deploy the new application version, then remove the old schema artifacts.:** Mitigation
- **Use a dedicated database migration job that runs before the ECS service update and is gated by a manual approval step in the workflow.:** Mitigation

5.2 Infrastructure Risks

5.2.1 Credential Exposure or Compromise

The AWS access key and secret stored in GitHub Secrets are a high-value target. A compromised key could allow an attacker to exfiltrate container images, manipulate ECS services, or escalate privileges within the AWS account.

- **Replace long-lived IAM access keys with GitHub Actions OIDC federation. Using OpenID Connect, GitHub Actions can assume an IAM role directly without storing any long-lived credentials in GitHub Secrets.:** Mitigation
- **If IAM keys are used, rotate them every 90 days and immediately revoke any key that may have been exposed.:** Mitigation
- **Enable AWS CloudTrail to log all API calls made using the deployment credentials for audit and incident response purposes.:** Mitigation

5.2.2 ECR Image Vulnerabilities

Container images can contain operating system packages with known CVEs. Deploying a vulnerable image to production exposes the application to security exploits.

- **Enable ECR image scanning with scan-on-push. Configure pipeline gates that fail the deployment if any critical or high severity CVEs are detected.:** Mitigation
- **Use minimal base images such as Alpine or distroless images to reduce the attack surface.:** Mitigation
- **Integrate a container scanning tool such as Trivy or Snyk into the GitHub Actions workflow as a pre-push step.:** Mitigation

5.2.3 ECS Task Placement Failures

ECS may fail to place new tasks if the cluster lacks sufficient CPU or memory capacity, if the task definition requests more resources than are available, or if Fargate service quotas are exhausted.

- **Monitor cluster CPU and memory reservation metrics in CloudWatch and set alarms to alert when utilisation exceeds 75%.:** Mitigation
- **Use ECS Service Auto Scaling to automatically add tasks in response to increased load, preventing the cluster from running at or near capacity under normal conditions.:** Mitigation

6. Deployment Considerations

Beyond risk mitigation, there are several operational and engineering considerations that directly impact the quality and maintainability of the deployment process.

6.1 Image Tagging Strategy

The current workflow tags all images as :latest, which provides no traceability — it is impossible to determine from the tag alone which version of the code a

running container is executing. For any production system, a more robust tagging strategy is essential.

- Recommended approach: tag images with both :latest and the Git commit SHA. The commit SHA tag (e.g. :sha-a3f5c12) allows precise identification of the deployed version and enables rollback to any previous commit by re-registering a task definition with the corresponding image URI.
- Semantic versioning tags (e.g. :v1.2.3) are appropriate for release-based workflows where deployment corresponds to a formal version increment.
- Environment-specific tags (e.g. :staging, :production) can be used in multi-environment pipelines to clearly identify which image is deployed where.

6.2 Rollback Procedure

The absence of a defined rollback procedure is a common deployment gap. If a bad deployment reaches production, the team must be able to revert quickly and confidently.

- Immediate rollback (task definition): Identify the previous ECS task definition revision number (e.g. my-app:42) and run aws ecs update-service --cluster <cluster> --service <service> --task-definition my-app:42 to force ECS to roll back to the prior version.
- Image-based rollback: If images are tagged with commit SHAs, the previous image URI can be used to register a new task definition revision that points to the known-good image, then update the service.
- Automated rollback: Configure CloudWatch alarms on key metrics (e.g. HTTP 5xx error rate, ECS task failure count) to trigger an automatic rollback action via an AWS Lambda function or CodeDeploy rollback hook.
- Document and rehearse: Rollback procedures should be documented, version-controlled alongside the deployment plan, and practised in a non-production environment to validate their effectiveness before they are needed in a real incident.

6.3 Environment Promotion

Production deployments should ideally be preceded by equivalent deployments to staging or pre-production environments. The current workflow deploys directly to production on every push to main. In a mature deployment pipeline, the following promotion model is recommended:

- Development: automatic deployment on every push to a feature or develop branch.
- Staging: automatic deployment on every merge to main, deploying to a staging ECS service and cluster that mirrors production.
- Production: triggered manually (via a GitHub Actions workflow_dispatch event or environment protection rule requiring a reviewer approval) after the staging deployment has been validated.

6.4 Observability and Monitoring

A deployment is not complete until monitoring is in place to detect problems in the newly deployed version. Without observability, degraded performance or errors may go undetected for extended periods.

- Logging: Configure the ECS task definition to use the awslogs log driver, which streams container stdout/stderr to Amazon CloudWatch Logs. Define a log group per service with a retention policy of at least 30 days.
- Metrics: Enable Container Insights on the ECS cluster to collect CPU, memory, network, and storage metrics at the task and container level. Publish custom application metrics to CloudWatch using the CloudWatch embedded metrics format (EMF).
- Alerting: Create CloudWatch alarms for key service indicators — error rate thresholds, task health check failures, high memory utilisation, and p99 latency. Route alarm notifications to an SNS topic connected to PagerDuty, Slack, or email.
- Distributed tracing: For microservices architectures, integrate AWS X-Ray to trace requests across service boundaries and identify performance bottlenecks.

6.5 Secrets Management

Application secrets (database passwords, API keys, third-party credentials) should never be baked into the Docker image or stored as plaintext environment variables in ECS task definitions. Use AWS-native secrets management to inject secrets securely at runtime.

- AWS Secrets Manager: Store secrets as key-value pairs and reference them in the ECS task definition using the secrets array. ECS retrieves and injects the secret value as an environment variable when launching the task.
- AWS Systems Manager Parameter Store: A cost-effective alternative for non-sensitive configuration and sensitive parameters (using SecureString type encrypted with KMS).
- Rotation: Enable automatic rotation for database credentials stored in Secrets Manager using the built-in Lambda rotation functions.

7. Scalability and Performance

The architecture built around ECS, ECR, and GitHub Actions is designed to scale horizontally with minimal architectural changes. This section outlines the scalability mechanisms available and the design principles that should guide capacity planning.

7.1 Horizontal Scaling with ECS Service Auto Scaling

ECS Service Auto Scaling allows the number of running tasks to be adjusted automatically in response to demand. Scaling policies are defined based on CloudWatch metrics and can be tuned to balance responsiveness against cost.

- Target Tracking Scaling: The simplest form of auto scaling. Define a target value for a metric (e.g. average CPU utilisation at 60%) and ECS automatically adds or removes tasks to keep the metric at the target.

- Step Scaling: More granular control — define different scaling increments for different metric threshold bands. For example, add 2 tasks when CPU exceeds 70% and add 5 tasks when CPU exceeds 90%.
- Scheduled Scaling: Pre-emptively scale out before known demand spikes (e.g. scheduled to increase task count ahead of peak business hours).
- Scale-in protection: Mark individual tasks as protected from scale-in to prevent ECS from terminating tasks that are currently processing long-running jobs.

7.2 Fargate vs EC2 Launch Type

The choice of ECS launch type has significant implications for scalability, cost, and operational overhead.

Launch Type	Characteristics
Fargate	Serverless. AWS manages the underlying infrastructure. Tasks scale out in seconds. No cluster capacity management required. Higher per-vCPU cost but lower operational overhead. Ideal for variable or unpredictable workloads.
EC2	You manage EC2 instances in the cluster. Lower cost at scale. Requires cluster capacity management, auto scaling groups, and instance patching. Suitable for large, predictable workloads where cost optimisation is critical.
Fargate Spot	Fargate tasks running on spare AWS capacity at up to 70% discount. Suitable for stateless, fault-tolerant workloads that can tolerate interruption. Mix with on-demand Fargate for cost-optimised, resilient services.

7.3 Load Balancing

For services that serve HTTP or HTTPS traffic, an Application Load Balancer (ALB) should sit in front of the ECS service. The ALB distributes incoming requests across all running task instances and performs health checks to route traffic only to healthy tasks.

- Connection draining: Configure deregistration delay on ALB target groups so that the load balancer stops sending new requests to a task before ECS terminates it during a rolling update, allowing in-flight requests to complete gracefully.
- Sticky sessions: If the application requires session affinity, enable ALB sticky sessions using duration-based or application-based cookies.
- Path-based and host-based routing: Use ALB listener rules to route traffic to different ECS services based on the request path or hostname, enabling multiple services to share a single load balancer.

7.4 Statelessness and Data Architecture

Containers are ephemeral and may be stopped, started, or replaced at any time by ECS. The application must be designed to be stateless — all persistent state must be stored externally.

- Session state: Store user sessions in Amazon ElastiCache (Redis) rather than in-process memory.
- File storage: Do not write files to the container filesystem. Use Amazon S3 for object storage or Amazon EFS for shared file system access.
- Database: Use Amazon RDS or Aurora with Multi-AZ enabled for high-availability relational data. Use connection pooling (e.g. RDS Proxy) to handle burst connections from scaled-out ECS tasks without exhausting the database connection limit.

7.5 CI/CD Pipeline Scalability

As the codebase and team grow, the GitHub Actions pipeline itself must scale to avoid becoming a bottleneck.

- Parallel test execution: Split the test suite into parallel jobs using a matrix strategy in GitHub Actions to reduce overall pipeline duration.
- Docker build caching: Use the actions/cache action or GitHub Actions cache backend for BuildKit to persist Docker layer cache between runs, significantly reducing build times for incremental changes.

- Self-hosted runners: For large organisations with high pipeline volumes, self-hosted GitHub Actions runners on EC2 or ECS can reduce per-minute costs and provide access to larger instance types with more CPU and memory.

8. Post-Deployment Checklist

After every deployment, the following checks should be performed to confirm that the new version is operating correctly before the deployment is considered complete.

- Confirm that the GitHub Actions workflow completed without errors and that all steps show a green status.
- Verify that the new image is present in ECR with the expected tag and that the image push timestamp matches the deployment time.
- Check the ECS service in the AWS console to confirm that the desired count equals the running count and that no tasks are in a PENDING or STOPPED state.
- Review CloudWatch Logs for the service log group and confirm that the application started successfully, with no FATAL or ERROR-level messages in the startup sequence.
- Execute a smoke test against the production endpoint — a basic HTTP health check or a representative API call — to confirm that the application is responding correctly.
- Check CloudWatch metrics for any anomalies in CPU utilisation, memory utilisation, error rates, or latency in the 5–10 minutes following deployment.
- Confirm that all CloudWatch alarms are in the OK state and that no alerts have fired since the deployment completed.

- Document the deployment in the team's deployment log, including the commit SHA deployed, the time of deployment, and the name of the person who triggered or approved it.

9. Glossary

Term	Definition
CI/CD	Continuous Integration / Continuous Deployment — the practice of automatically building, testing, and deploying code changes.
ECR	Amazon Elastic Container Registry — a managed Docker container registry service provided by AWS.
ECS	Amazon Elastic Container Service — a container orchestration service that manages the deployment and scaling of Docker containers on AWS.
Fargate	A serverless compute engine for containers that removes the need to provision or manage EC2 instances for ECS tasks.
IAM	AWS Identity and Access Management — the service used to control access to AWS resources via users, roles, and policies.
Task Definition	An ECS configuration template describing how a container should run, including image, CPU, memory, ports, and environment variables.
Rolling Update	A deployment strategy in which running instances are replaced incrementally with new versions without taking the service offline.
Blue/Green	A deployment strategy that uses two identical environments to enable zero-downtime deployments and instant rollbacks.
Canary Release	A gradual rollout strategy that routes a small percentage of traffic to the new version before full deployment.
ALB	Application Load Balancer — an AWS load balancing service that routes HTTP/HTTPS traffic across multiple targets.
CVE	Common Vulnerabilities and Exposures — a list of publicly known security vulnerabilities in software packages.

KMS	AWS Key Management Service — a managed service for creating and controlling encryption keys.
OIDC	OpenID Connect — a protocol used to federate identity between GitHub Actions and AWS IAM without long-lived credentials.